Tunisian Republic
Ministry of Higher Education and Scientific Research

University of Tunis
Tunis Business School

Summer Internship Report

As a consolidation of the student's Junior Year in Business Analytics

AML Name Screening System

Realized by: Ahmed Mnaouer
Host company: TALYS Consulting

THE HOUSE OF DIGITAL EXPERTS

Academic Supervisor: Dr. El Moubarki Lassad
Major: Business Analytics
Minor: Information Technology

Academic year 2024–2025

Abstract

This report presents an Arabic-aware, fully offline Anti-Money Laundering (AML) name screening system built in Python.

The system screens customer names against a sanctions and politically exposed persons (PEP) watchlist, combining a fast fuzzy rule-based engine with a supervised machine learning layer for three-way decisions (`ALLOWED`, `AMBIGUOUS`, `BLOCKED`).

It is implemented as a dual-engine backend with a Flask REST API and a Streamlit user interface, and is designed so that all processing occurs locally on the institution's infrastructure.

The report describes the data, methodology, system architecture, implementation choices, and empirical performance, and concludes with recommendations for further improvements.

Contents

Abstract **1**

1 Introduction **1**

2 Data Description **1**
2.1 Watchlist Dataset 1
2.2 Generated Training Dataset 2

3 Methodology and System Design **3**
3.1 Overall Architecture 3
3.2 Rule-Based Fuzzy Matching Engine 4
3.3 Training Data Generation and Labeling 5
3.4 Supervised Models 5
3.5 API Backend and Transliteration 6
3.6 Streamlit Frontend 6

4 Implementation **6**
4.1 Feature Definition 6
4.2 Example: Combined Score Computation 7
4.3 Example: XGBoost Model Setup 7

5 Results and Analysis **8**
5.1 Hold-Out Test Performance 8
5.2 Cross-Validation 9
5.3 Feature Importance 9

6 Recommendations and Improvements **10**

7 Conclusion **10**

List of Figures

1 Risk category distribution in the watchlist dataset. 1
2 Distribution of generated decision labels in the training dataset. 3
3 Distributions of Jaro-Winkler and Levenshtein scores by decision class in the generated training dataset. 3
4 High-level system architecture: watchlist preprocessing, dual engines, API backend, and Streamlit frontend. 4
5 ROC curves for all models and decision classes. 8
6 Confusion matrices for supervised models on the test set. 9
7 Relative feature importance for the XGBoost model. 9

1 Introduction

Financial institutions are required to screen customer and counterparty names against regulatory watchlists as part of their AML and counter-terrorism financing obligations. This screening is challenging in multilingual environments, particularly when names can appear in Arabic script, Latin transliterations, or noisy variants with typos, reordered tokens, or phonetic differences. This system was developed during a summer internship at TALYS Consulting as part of an AI & Machine Learning project, supervised by Dr. El Moubarki Lassad.

2 Data Description

2.1 Watchlist Dataset

The core input is a watchlist stored in `cleaned_aml_data.xlsx`.

- `Full Name`: Name of the person or entity, possibly in Arabic or Latin script.
- `Risk Category`: Categorical label.

Table 1: Representative sample of watchlist entries

Full Name	Risk Category	Nationality	Residence	Source of Funds	Flagged
روفيدا كانو	Terrorism Financing	China	France	Real Estate	Yes
Elizabeth King	Money Laundering	France	Turkey	Donations	Yes
Melissa Hernandez	PEP	Saudi Arabia	Spain	Government Salary	No
جلال الدّين آل جعفر	Money Laundering	Jordan	France	Government Salary	Yes
Étienne Le Gall	PEP	Jordan	Spain	Offshore Funds	No
Nicole Moore	Terrorism Financing	Nigeria	France	Government Salary	Yes

During preprocessing in the basic engine (`screening_system.py`), rows with missing names or risk categories are dropped to ensure that every watchlist entry can contribute to similarity comparisons and downstream risk assessment. Date-formatted birthdates are converted to integer ages when possible to normalize the age field.

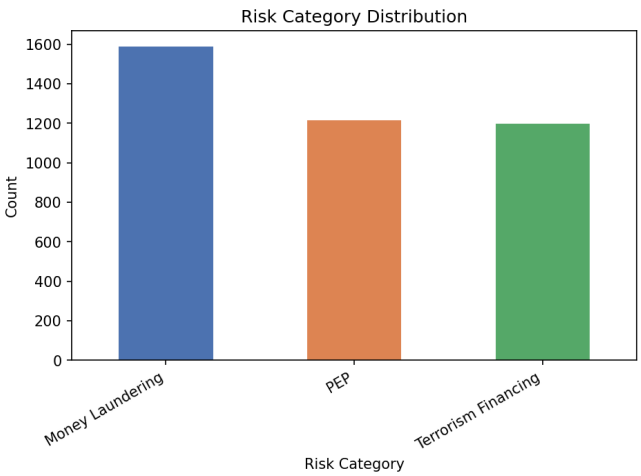

Figure 1: Risk category distribution in the watchlist dataset.

2.2 Generated Training Dataset

The enhanced engine (`screening_system_enhanced.py`) does not train directly on raw names. Instead, it constructs a synthetic training dataset using the `AMLTrainingDataGenerator` class, which generates realistic pairs of (input name, watchlist match) along with derived features and weak labels.

For each watchlist entry, the generator creates variations of the original name by introducing character substitutions, deletions, and word-order changes (e.g., reversed or shuffled tokens). It also creates random names from generic first and last name lists to represent non-matching background cases. For each generated input name, the system:

1. Finds the best matching watchlist entry based on a combined fuzzy similarity score.
2. Derives feature values such as fuzzy similarity metrics and binary risk flags.
3. Applies rule-based decision logic (`ALLOWED`, `AMBIGUOUS`, `BLOCKED`) to obtain a label.
4. Introduces controlled label noise on a small proportion of samples to improve robustness.

The final training DataFrame contains, among others, the following columns:

- `input_name`
- `jaro_winkler_score`
- `levenshtein_score`
- `soundex_match`
- `is_high_risk`, `is_medium_risk`, `has_nationality_match`
- `combined_score`
- `risk_category`
- `decision` (string: `ALLOWED`, `AMBIGUOUS`, `BLOCKED`)

Class balancing is achieved via resampling so that each decision category is represented with approximately equal frequency in the final training set.

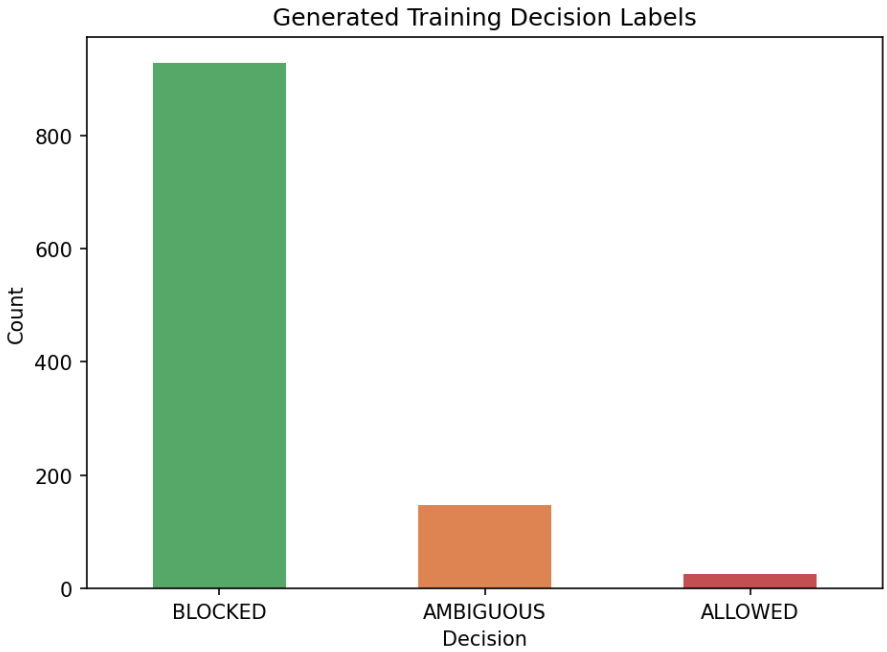

Figure 2: Distribution of generated decision labels in the training dataset.

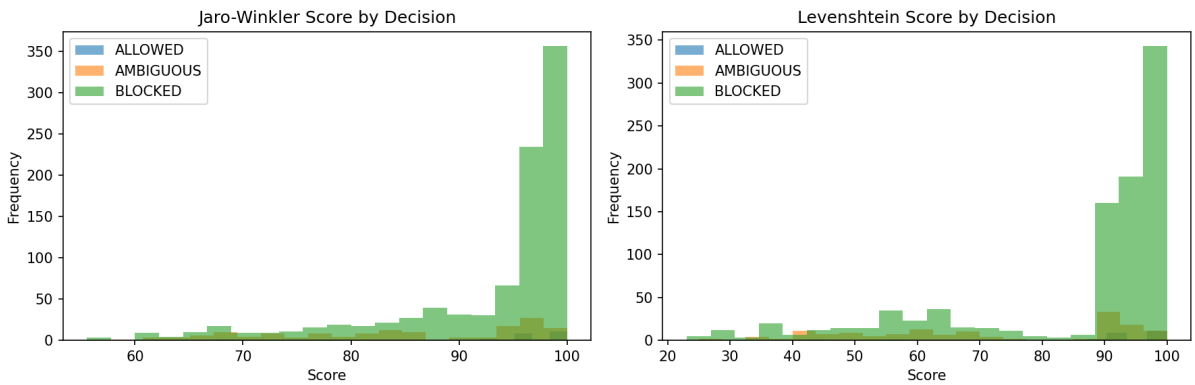

Figure 3: Distributions of Jaro-Winkler and Levenshtein scores by decision class in the generated training dataset.

3 Methodology and System Design

3.1 Overall Architecture

The project implements a dual-engine AML name screening architecture. At a high level, the components are:

- **Rule-Based Engine** (`screening_system.py`): CLI-oriented pipeline that loads the watchlist, performs fuzzy name matching using Jaro–Winkler, Levenshtein, and Soundex, and applies heuristic decision rules.
- **ML Engine** (`screening_system_enhanced.py`): Data generation and model training pipeline producing supervised classifiers that map engineered features to decisions.

- **API Backend** (`api_backend.py`): Flask application that exposes a `/predict` endpoint, handles custom Arabic-to-Latin transliteration, computes fuzzy features, and uses the trained ML model and scaler to produce a decision.
- **Frontend** (`app_streamlit.py`): Streamlit user interface that allows analysts to enter names, call the backend API, and display similarity scores, risk category, and final decisions with explanations.

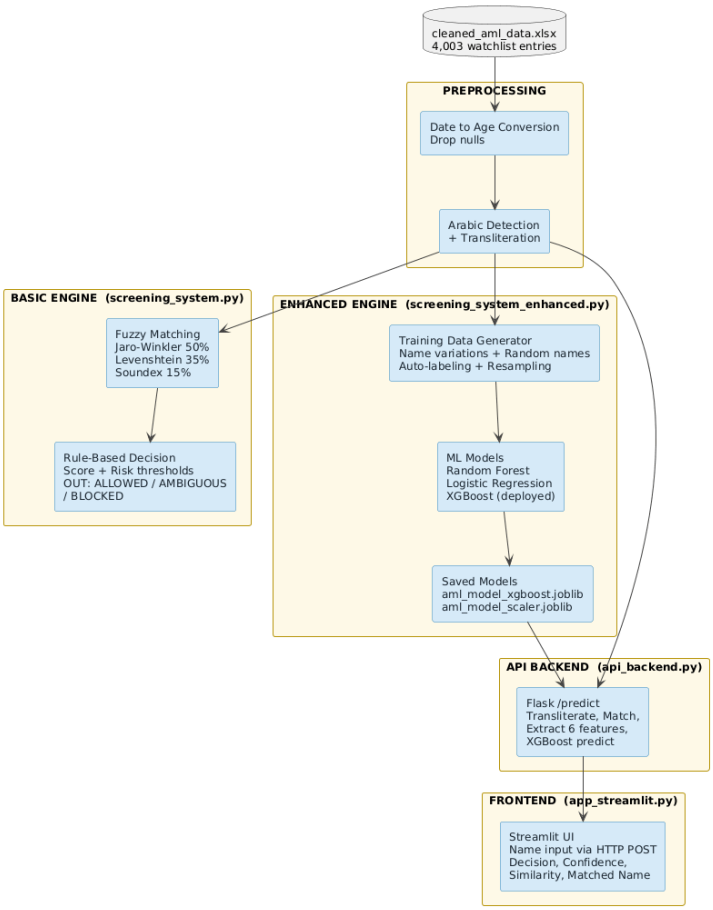

Figure 4: High-level system architecture: watchlist preprocessing, dual engines, API backend, and Streamlit frontend.

3.2 Rule-Based Fuzzy Matching Engine

In the basic engine, each input name is compared against all names in the watchlist. For a given pair of normalized strings n_1 and n_2, the following similarity components are computed:

- Jaro–Winkler similarity (JW), scaled to a 0–100 range.
- Normalized Levenshtein similarity (Lev), computed as $(1 - \text{distance}/\max(|n_1|, |n_2|)) \times 100$.
- Binary Soundex match (SX), with value 100 if the Soundex codes match and 0 otherwise.

The combined similarity score is a weighted average:

$$\text{Score} = 0.5 \cdot \text{JW} + 0.35 \cdot \text{Lev} + 0.15 \cdot \text{SX}.$$

The best match in the watchlist is the record with the highest combined score. This match is associated with a `Risk Category`, which is mapped into a coarser `risk_level` (`High Risk`, `Medium Risk`, `Unknown Risk`). The engine then applies threshold-based rules on the similarity score and risk level to obtain a decision (`ALLOWED`, `AMBIGUOUS`, or `BLOCKED`).

3.3 Training Data Generation and Labeling

For the machine learning engine, training data is created using the `AMLTrainingDataGenerator` class. Name variations (typos, deletions, word-order changes, partial names) simulate realistic user input such as misspellings or shortened names. Random names simulate non-matches drawn from a generic population.

For each generated input name, the generator:

1. Computes fuzzy features (`jaro_winkler_score`, `levenshtein_score`, `soundex_match`) against the best-matching watchlist row.
2. Constructs binary indicators for whether the best match belongs to a high-risk or medium-risk category, and whether nationality co-occurs in the input.
3. Computes a `combined_score` as a weighted linear combination of the fuzzy scores and nationality indicator.
4. Applies decision rules analogous to the basic engine to assign `decision` labels.
5. Optionally introduces label noise by flipping a small percentage of labels to simulate human annotation errors and reduce overfitting.

After generation, the dataset is balanced by upsampling each decision class to a common target size.

3.4 Supervised Models

The `AMLMLModels` class wraps model training and evaluation. The following models are trained on the same standardized feature set:

- Random Forest classifier with depth, split, and leaf constraints and `class_weight="balanced"`.
- Multinomial Logistic Regression with class weighting and regularization.
- XGBoost classifier with early stopping, regularization terms (`reg_alpha`, `reg_lambda`), and a moderate learning rate.

Features are standardized using `StandardScaler`. For XGBoost, an internal validation split is created for early stopping based on multiclass log-loss. All models are later evaluated on a held-out test set and via cross-validation.

3.5 API Backend and Transliteration

The Flask backend loads:

- The original watchlist DataFrame.
- A transliterated copy where Arabic names are converted to Latin using a custom character map.
- The trained XGBoost model and its associated scaler from `joblib` files.

Incoming names are normalized (lowercased, whitespace-normalized, punctuation stripped, simple diacritics removed) and then compared against the transliterated watchlist to find the best match. Fuzzy similarity features and risk flags are computed; the scaled feature vector is passed to the XGBoost model to obtain a probability distribution over decisions, which is then mapped to `ALLOWED`, `AMBIGUOUS`, or `BLOCKED`. The backend returns a JSON response with the decision, confidence, similarity percentage, top match details, and a short reason string.

3.6 Streamlit Frontend

The Streamlit frontend provides an analyst-friendly interface where the user can:

- Enter one or more names to screen.
- Trigger backend predictions via HTTP requests.
- View similarity scores, top-matching watchlist records, risk categories, and the model's decision and confidence.

The frontend focuses on clarity and immediate feedback rather than complex visualizations, but can be extended with additional charts as needed.

4 Implementation

Given the size of the codebase, this report includes only small representative snippets; the full source is available in the `backend/` and `frontend/` directories. The emphasis here is on explaining design and behavior rather than reproducing entire scripts.

4.1 Feature Definition

The ML engine uses a fixed feature set derived from fuzzy similarity and risk flags:

Listing 1: Feature columns used for supervised models.

```
self.feature_columns = [
    "jaro_winkler_score",
    "levenshtein_score",
    "soundex_match",
    "is_high_risk",
    "is_medium_risk",
    "has_nationality_match",
]
```

4.2 Example: Combined Score Computation

The training data generator computes a combined score that aligns with the logic of the basic engine:

Listing 2: Illustrative combined score calculation.

```
combined_score = (
    features["jaro_winkler_score"] * 0.4
    + features["levenshtein_score"] * 0.3
    + features["soundex_match"] * 100 * 0.1
    + features["has_nationality_match"] * 100 * 0.2
)
```

4.3 Example: XGBoost Model Setup

The XGBoost classifier is configured with explicit regularization and early stopping on a validation set:

Listing 3: Illustrative XGBoost configuration.

```
xgb_model = xgb.XGBClassifier(
    n_estimators=1000,
    max_depth=3,
    learning_rate=0.05,
    reg_alpha=0.1,
    reg_lambda=1.0,
    min_child_weight=5,
    random_state=42,
    eval_metric="mlogloss",
)

early_stop = xgb.callback.EarlyStopping(
    rounds=50,
    metric_name="mlogloss",
    save_best=True
)

xgb_model.fit(
    X_tr, y_tr,
    sample_weight=class_weights[y_tr],
    eval_set=[(X_val, y_val)],
    callbacks=[early_stop],
    verbose=False,
)
```

These examples illustrate the level of control over feature engineering and model regularization without overwhelming the reader with boilerplate.

5 Results and Analysis

5.1 Hold-Out Test Performance

Each model is evaluated on a held-out test set using:

- Overall accuracy.
- Macro-averaged F1-score across the three classes.
- Multiclass ROC-AUC (one-vs-rest).
- Macro-averaged Precision–Recall AUC.

Model	Accuracy	Macro F1	ROC-AUC (ovr)	PR-AUC (macro)
Random Forest	80.43%	80.57%	95.11%	90.67%
Logistic Regression	63.38%	63.18%	78.21%	59.28%
XGBoost	80.43%	80.44%	94.67%	89.56%

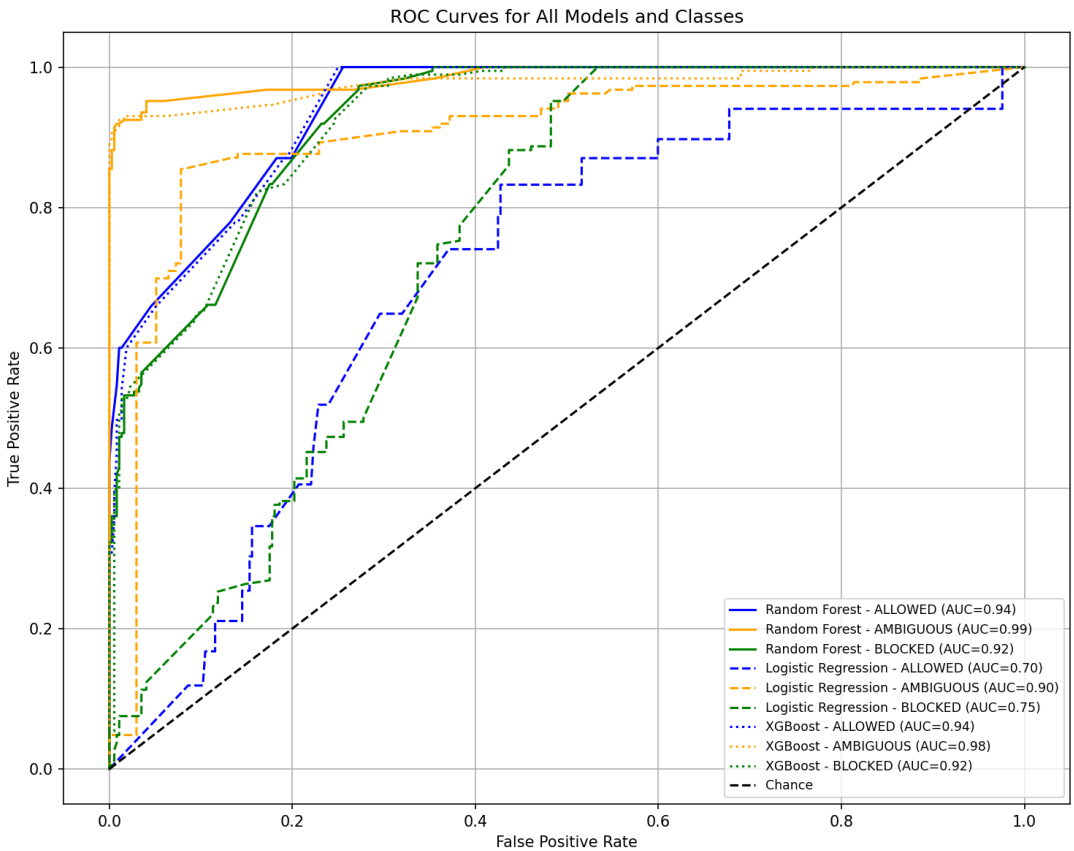

Figure 5: ROC curves for all models and decision classes.

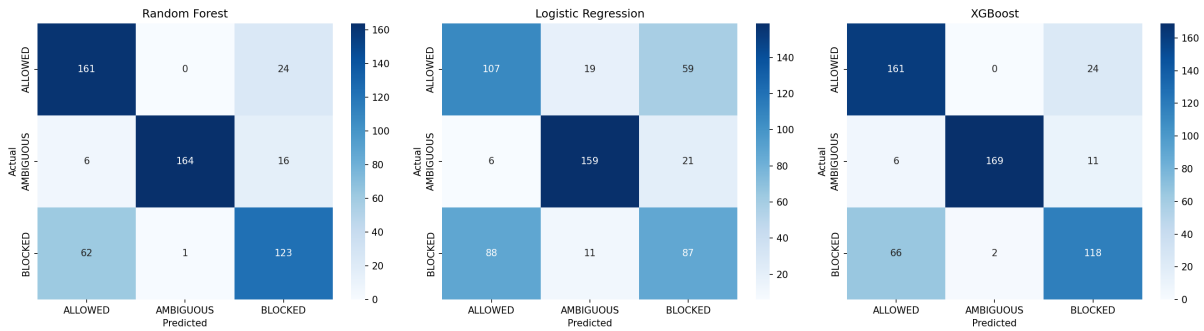

Figure 6: Confusion matrices for supervised models on the test set.

5.2 Cross-Validation

To assess stability, 5-fold stratified cross-validation is run on the training set for each model, using accuracy as the scoring metric. The mean and standard deviation of cross-validation accuracy provide an indication of variance due to sample splits.

5-fold stratified cross-validation on the training set yields the following results: Random Forest achieves a mean CV accuracy of 77.86% (± 0.85%), XGBoost achieves 77.28% (± 1.56%), and Logistic Regression achieves 62.51% (± 2.03%). The low standard deviations for Random Forest and XGBoost indicate stable learning across folds, while Logistic Regression shows slightly higher variance, consistent with its weaker overall performance on this non-linearly separable feature space.

5.3 Feature Importance

In the trained XGBoost model, `is_high_risk` is by far the most influential feature (importance score 0.51), followed by `has_nationality_match` (0.20) and `jaro_winkler_score` (0.10). This indicates that the model relies heavily on the risk category of the best-matching watchlist entry, using fuzzy string scores as secondary confirmation. `is_medium_risk` receives zero importance, as its signal is captured by the interaction between `is_high_risk` and the similarity features.

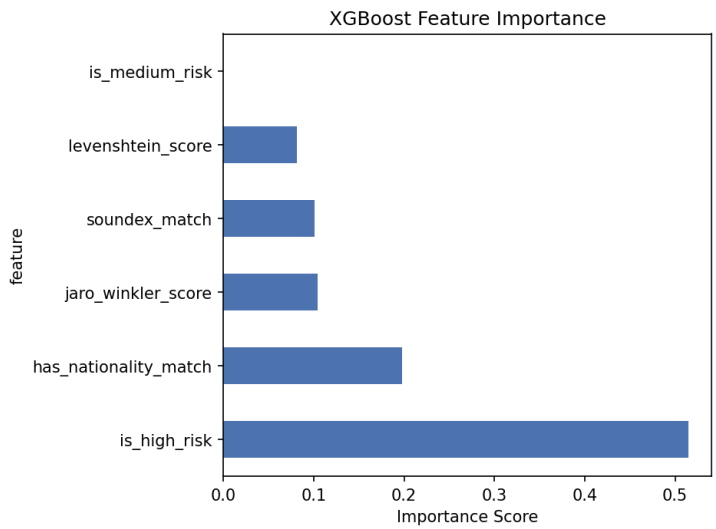

Figure 7: Relative feature importance for the XGBoost model.

6 Recommendations and Improvements

Based on the implemented system and available results, several enhancements are recommended:

- **Calibration and Thresholding.** Calibrate predicted probabilities (e.g., via Platt scaling or isotonic regression) and explicitly optimize decision thresholds for `ALLOWED`, `AMBIGUOUS`, and `BLOCKED` under institution-specific risk preferences.
- **Expanded Features.** Incorporate additional data fields (e.g., date of birth similarity, address, document identifiers) where available to reduce false positives in common-name scenarios.
- **Explainability.** Extend the API to return feature contributions (e.g., SHAP values or simpler score breakdowns) to support analyst understanding of why a name was blocked or sent for review.
- **Operational Monitoring.** Log screening outcomes, manual overrides, and analyst feedback in a structured way, and periodically retrain models using updated labeled data.
- **Robust Transliteration.** While the current custom transliteration map is zero-dependency and effective for many Arabic names, further refinement or integration with specialized transliteration libraries could improve coverage and reduce edge cases.
- **Scalability.** For very large watchlists, consider approximate nearest-neighbor search or blocking strategies on phonetic keys to reduce the computational cost of full pairwise comparisons.

7 Conclusion

This mini project demonstrates an end-to-end AML name screening system that combines fuzzy string matching, synthetic data generation, and supervised machine learning into a coherent architecture. The rule-based engine provides interpretable baselines and decision rules, while the ML engine leverages engineered features to refine decisions based on empirical performance. A Flask API and Streamlit frontend wrap these components into a user-facing tool that can operate fully offline, which is often a practical requirement in regulated environments. Future work can focus on integrating richer customer data, improving model calibration and explainability, and embedding the system into a broader AML workflow with auditability and feedback loops.